UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Ingegneria

Corso di Laurea in Ingegneria Informatica

Thesis

Probabilistic Soft Real-time Analysis

Edoardo Sarri

7173337

Ottobre 2026

Contents

1 Simulator **5**

1.1 Taskset Model 5

1.2 Input and Output 6

1.2.1 Output - Logging 6

1.2.2 Output - Distribution 6

1.2.3 Output - Deadline Miss 7

1.3 Implementation 7

1.3.1 Utilities 8

1.3.2 Flow 9

1.4 Distribution Extraction 10

1.4.1 Java 10

1.4.2 Python 12

1.5 Testing 12

1.5.1 System Test 12

1.5.2 Stochastic Test 12

2 Bernstein Polynomials Library **13**

2.1 Library Constraints 13

2.2 Approximation of Empirical Cumulative Distribution Function 14

2.2.1 Implementation 14

2.3 Approximation of Polynomial 14

2.3.1 Implementation 15

2.3.2 Approximation Techniques 15

2.3.3 Functionality 16

2.3.4 Distance 16

3 Experiments **17**

3.1 Monomial Approximation 17

List of Figures

1.1 Simulator Class Diagram. 8
1.2 Simulator Activity Diagram. 11

2.1 Bernstein Polynomials Library Class Diagram. 13

Listings

1 Simulator

The simulator's general objective is to have an instrument to validate our ideas and results in an empirical way. In particular, we want to obtain experimentally for each task, the distribution of job execution time.

This kind of simulation must be totally deterministic; for this reason all timing aspects have been handled in a discrete way using the `Duration` Java type. In this way we can handle the complexity of small durations with a high abstraction level. We assume that the time is taken as input and returned as output in milliseconds. During the execution, to obtain the maximum precision, it's processed in nanoseconds.

1.1 Taskset Model

In this section we define the taskset model, i.e., how the tasks in the taskset can be defined and what are their non-functional characteristics. From now on we assume that:

- We assume we have a single core CPU, which implies that we are working with a concurrent problem.
- For tasks:
  - Each task is independent of the others. This means that there aren't shared resources among two tasks.
  - Tasks can be both periodic and sporadic (i.e. the releases between two consecutive jobs of the same task are separated by a minimum inter-arrival time). For the sporadic task, each job release time can be sampled from some distribution.
  - Each task has a single elementary execution unit, i.e. each task has only one chunk.
  - The task execution time is defined probabilistically, i.e. the execution time of each job of a task is sampled from some distribution. These distributions can include also the deterministic ones.
  - The deadline of each task is known a priori.
  - The first release time is by default at time 0, but it can be sampled from some distribution as well.
- The job execution times are i.i.d. (i.e., independent and identically distributed).

This means that only one distribution represents the job execution time pattern of a given task.

1.2 Input and Output

The simulator inputs are the following:

- A fixed priority scheduling algorithm. As we will see in Section 1.3 the scheduling algorithm implemented is Deadline Monotonic: we can handle periodic and aperiodic tasks with a fixed priority.
- The taskset, characterized by the assumptions in Section 1.1.
- The duration of the simulation.
- The strategy for the deadline miss policy. We have the following option:
  - **Incomplete Jobs Aborted (JA)**: We abort the job that is responsible for the deadline miss.
  - **Jobs run to Completion (JC)**: We continue the simulation with the job that has exceeded its deadline.
  - There is also the policy where we abort the whole simulation if we discover a deadline miss.

All these aspects can be passed through the `Main.java` source file or by the `config.yaml` or `config.json` file. The second requires the code for the handling of the mapping from the config file to the Java object; this code is a simple call to the already implemented `ConfigLoader.java` class. This handling is made with the *Jackson* library, that is responsible for the serialization of the configuration in Java objects.

1.2.1 Output - Logging

To test the scheduler algorithm implementation and to have evidence of the system behavior, the simulator can produce a file, named *trace.log*. This file contains a sequence of <*time, event*>, where *event* can be: the release of a job, the execution and the completion of a job, the task preemption and the deadline miss event.

1.2.2 Output - Distribution

During the simulation the system retrieves all execution times sampled by all tasks using the `TaskExecutionTimeCollector` class. These data are sent to a Python script that is the only one responsible for dealing with them. The final output is a distribution of execution time for each task.

1.2.3 Output - Deadline Miss

As we mentioned before, the simulator can identify a deadline miss event. This doesn't imply the abort of the whole simulation, but only the abort of the job that has missed its deadline; in this way the simulator can go on. In each case of the deadline miss we use an event collector that stores the number of aborted jobs on the number of released jobs, for each task. In this way at the end of the simulation we can extract a statistic about the aborted jobs.

1.3 Implementation

In Figure 1.1 is shown the class diagram of the simulator. In the following we present the core entities that compose the system.

- **Client**
  Is the Main of the Java system. It is responsible for allocating the taskset and the scheduler, calling `analyze()` on the scheduler and the `callPythonExtractor` of the `MyUtils` class.

- **TaskSet**
  Is a simple set of all tasks.

- **Task**
  Represents a task. It models only the static specification of the task: the period, the execution time and the deadline; the first two are Sirio `Sampler` objects that represent the distribution from which they sample the true duration; the last is a `Duration` object, where the time must be expressed in milliseconds. Internally it also maintains a `jobCounter` to uniquely identify its jobs.

- **Job**
  Represents a task instance. While a Task contains only the task specifications, a Job contains the active state: the absolute deadline, the remaining execution time, the information about its completeness and the time of completion. It has a method to handle the simulation of its execution.

- **Scheduler**
  Represents the heart of the simulation, which uses the taskset for the scheduling simulation. It's responsible for initializing the system clock with a `MyClock` object and handling the system logging and the simulation duration.

- **FixedPriorityScheduler**
  Represents the abstraction for all fixed priority scheduling algorithms. All its subclasses must only implement the assign priority method.

- **DeadlineMonotonicScheduler**

Is the specialization of the `FixedPriorityScheduler` that implements the priority assignment of the Deadline Monotonic scheduling algorithm.

- **Event**
  Is the abstract class that represents the relevant events of the system; each event is defined by the time of its occurrence. The idea is that the time can go forward until it reaches an instance of this class. The two implementations are: `ReleaseEvent` which models the job release event; `DeadlineEvent` which models the reaching of the job deadline.

- **DeadlineMissStrategy**
  Is the interface responsible for the implementation of the jobs deadline miss policy. There are the possibility: the `AbortSimulationStrategy` that interrupts the whole simulation; the `AbortJobStrategy` that aborts the job and continues the simulation; the `ContinueStrategy` that do nothing and leave the job in the ready job queue. All these three solution the job abortion and collect the related statistics.

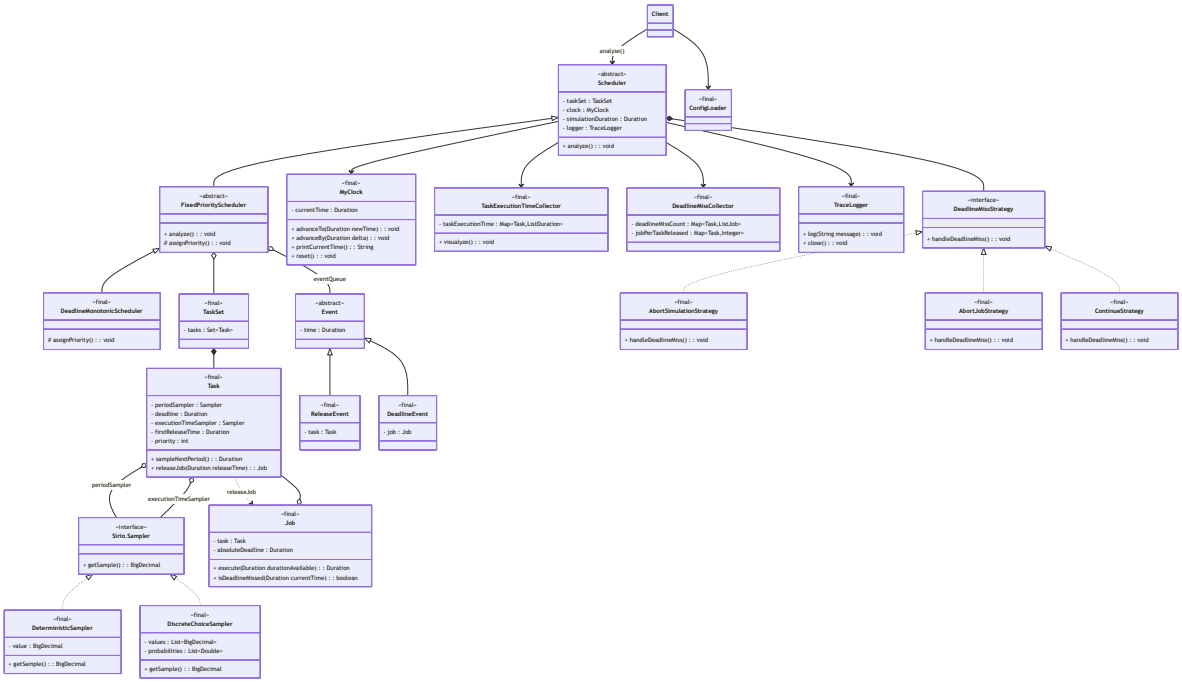

Figure 1.1: Simulator Class Diagram.

1.3.1 Utilities

If we observe Figure 1.1 we have to describe the system utilities.

- **ConfigLoader**
  It is the class responsible for the mapping from the `config.yaml` file to the Java objects.

- **Sirio.Sampler**
  It is the Sirio interface. We have defined two implementation:
  - **DeterministicSampler**
    It always returns a constant value. Useful for deterministic parameters that allow also probabilistic values.
  - **DiscreteChoiceSampler**
    Represent a weighted choice.
- **MyClock**
  Represents the global clock of the system. Implements the method to go at a specified time, to go forward in time by a specified duration and to print the current time.
- **TraceLogger**
  The logging utils that is responsible for logging the scheduling main events in a file named *trace.log* with a personalized format. An interface that abstracts the logging operations.
- **TaskExecutionTimeCollector**
  It is the collector of all execution times of all tasks. It allows the visualization of the empirical distribution (ECDF and EPDF).
- **DeadlineMissCollector**
  It is the collector of all aborted jobs of all tasks. This is useful to have a trace of all jobs that have been aborted due to deadline misses, to analyze them in a post-mortem way.

1.3.2 Flow

As illustrated in Figure 1.2, the scheduler flow follows these steps:

1. Assign task priorities according to the scheduling algorithm.
2. Reset the system state. This allows the client (i.e., `Main.java`) to call `analyze()` multiple times for consecutive simulations on the same instance.
3. Release the first job for each task in the taskset.
4. Iterate through the events in the queue until the next event time exceeds the simulation duration.
   a) Retrieve the first event from the queue and calculate the available time between the current clock and the event time.
   b) Distribute this available time among the ready jobs according to their priority.

i. Access the highest priority job from the ready queue.

ii. Check if the last executed job must be preempted by the current highest priority job for logging purposes.

iii. Execute the job for the available time. If the job finishes, remove it from the ready list.

iv. Advance the system clock by the actual execution time and check for deadline misses.

v. Update the remaining available time.

c) Synchronize the system clock with the event time and check for deadline misses for all ready jobs to ensure the state is consistent.

d) Process all events that occur at the same time as the current clock. For a new release: create the job, sample its execution time, and add both its deadline and its next release time to the event queue.

5. If the simulation duration has not been reached, execute the simulation for the remaining time.

1.4 Distribution Extraction

Since the main objective of the whole system is to produce the distribution of the tasks execution time, we have implemented a Python script that can be launched directly from `Main.java` with a java call, immediately after the generation of the simulation data. This prevents us from jumping from a Java system to a Python script: we only have to run the simulator system.

The Python script represents the information retrieved during the simulation by the `TaskExecutionTimeCollector` collector in a *.pdf* file in *results/* directory.

1.4.1 Java

The Java `Main` creates a `ProcessBuilder` to execute directly the Python script and also redirects the console and the bash: now Python has full control.

Once the simulator has produced the data, they are stored in a `Map` object. Starting from this map, we use the *Jackson* library to map data to a JSON file and transfer them through a pipe in an efficient way without writing any new code; the transfer is made through the Python script's standard input, to avoid the generation of temporary files that clutter the project file system.

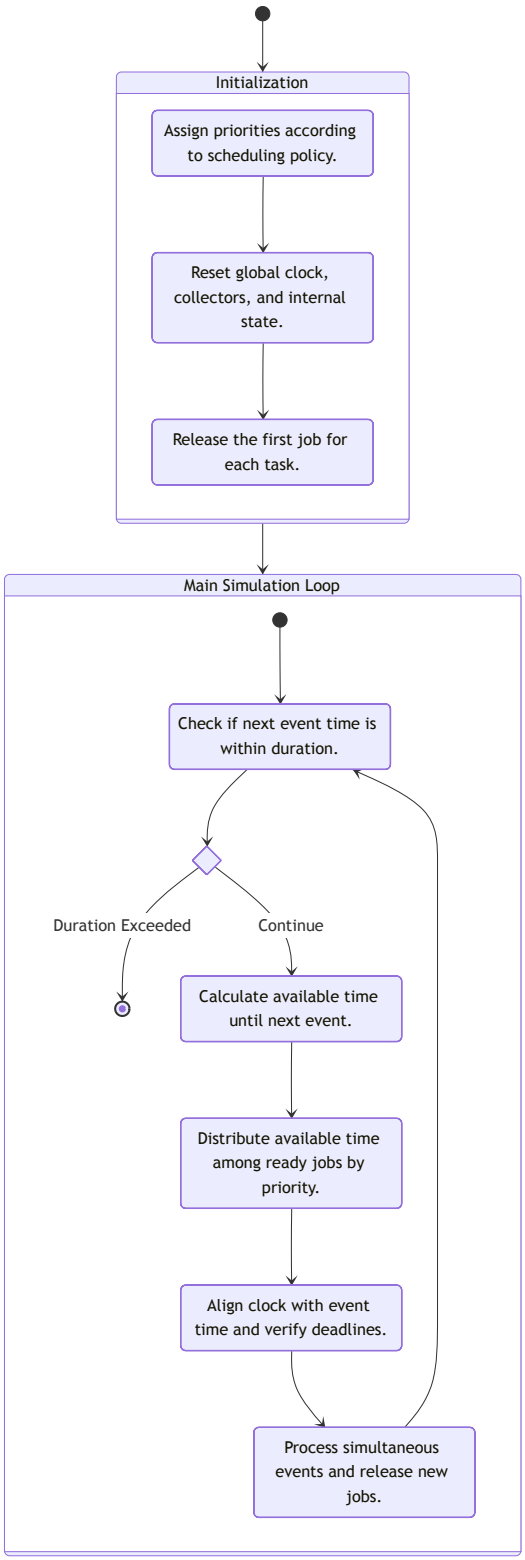

Figure 1.2: Simulator Activity Diagram.

1.4.2 Python

Once Python has read the whole data, it converts the JSON file into a dictionary. At this point, it is very simple to obtain the distribution of the execution time of all tasks.

1.5 Testing

The simulator tests are divided into two categories: the deterministic system test and the stochastic test.

1.5.1 System Test

This kind of test always uses the `DeterministicSampler`. The objective is to verify the correct behavior of the simulator system in various scheduling scenarios.

The scenarios that were tested are: single task execution; properties of the DM scheduling; task preemption (including nested preemption); overload of the system, i.e., execution of a taskset with a load factor greater than 1 resulting in aborted jobs; idempotency of the analysis process (i.e., multiple consecutive calls of `analyze()` produce the same result); offset management for tasks with non-zero initial release times; handling of priority collisions; boundary cases such as jobs completing exactly at their deadline or tasks with zero execution time; semantic validation of the traces; correct handling of all possible scenario on deadline miss discoverty.

These tests guarantee: the correctness of the DM scheduling algorithm; the precision of the clock implementation; the reliability of reusing the same instance for state handling; and the robustness of the engine in complex scheduling scenarios.

1.5.2 Stochastic Test

This kind of simulation uses real stochastic distributions to sample the parameters of the jobs.

The scenarios tested are the following: repeated simulations with different stochastic distributions (e.g., `UniformSampler` and `ExponentialSampler`) and multiple tasks; engine robustness with high task density and random loads to ensure simulation stability.

2 Bernstein Polynomials Library

In Figure 2.1 we can see the architecture behind the Bernstein Polynomials Library.

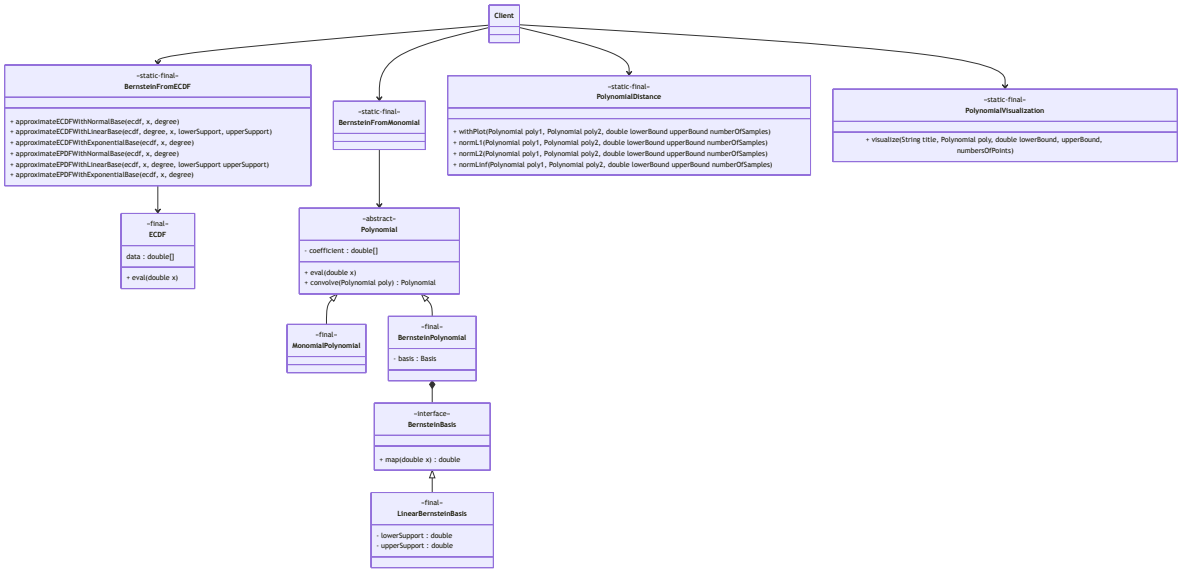

Figure 2.1: Bernstein Polynomials Library Class Diagram.

2.1 Library Constraints

The critical part of the library, from a computational perspective, is the calculation of the binomial coefficients. The whole library is based on `double` type and this implies some limitations on the degree of the polynomial that can be evaluated.

These limitations are due to two factors:

- Range
  The exponent allows us to represent numbers up to 1.8×10^{308}.
- Precision
  We can guarantee a precision up to 15 decimal digits: the 52 digits of the mantissa brings the machine ϵ error to circa 10^{-16}.

The consequence of this approach is the following:

- Within a $degree < 54$ we can compute the polynomial without any precision loss.
- If the degree is within the $[54, 1029]$ interval we have a precision loss due to the large binomial coefficients and small powers, but we can still compute the

polynomial.

- For $degree \geq 1030$ we have an overflow of the binomial coefficients and the polynomial cannot be computed.

2.2 Approximation of Empirical Cumulative Distribution Function

In this Section we discuss the part of the library responsible for the approximation of a Empirical Cumulative Distribution Function (ECDF) with a Bernstein polynomial.

The system allows us to approximate both CDF and PDF curve, given an empirical CDF F. For each curve we have three variants of basis:

- **Normal**: The original ECDF must be in $[0, 1]$, i.e. $F : [0, 1] \to [0, 1]$.
- **Linear**: The original ECDF must be in $[a, b]$, i.e. $F : [a, b] \to [0, 1]$, where both a and b must be finite value.
- **Exponential**: The original ECDF can be in $[0, +\infty]$, i.e. $F : [0, +\infty] \to [0, 1]$.

2.2.1 Implementation

From the Figure 2.1 we can see that there are only two single classes that work together:

- **BernsteinFromECDF**
  It is the facade of the system. It exposes six methods, three for the approximation of the empirical CDF and three for the empirical PDF.
- **ECDF**
  It is the representation of the empirical CDF. It allows the evaluation of the ECDF in a point x.

2.3 Approximation of Polynomial

The system allows us to map a polynomial in monomial form to a Bernstein polynomial. In particular, given a polynomial $P(x) = \sum_{i=0}^{M} a_i x^i$ of degree M with known analytical form, we want to identify the Bernstein Polynomial $B(x) = \sum_{i=0}^{N} \beta_i b_{i,N}(x)$ of degree N that best fits it.

2.3.1 Implementation

In Figure 2.1 there is the subsystem that is responsible for the approximation from the polynomial defined by monomial to a Bernstein Polynomial. Let's see the main components:

- **BernsteinFromMonomial**
  It's the facade to use the system and abstract the domain model that shape the polynomial.
- **Polynomial**
  It's the abstract class that wraps the state of a polynomial, i.e. the coefficients that define the polynomial. It defines also the `eval` method to evaluate the polynomial in given point `x` and the method `convolve()` to perform the convolution between two polynomials of the same type.
- **MonomialPolynomial**
  Classical polynomial where the basis is defined as an integer power of `x`.
- **BernsteinPolynomial**
  It represents the Bernstein polynomial. It's defined by the coefficient and the type of `Basis`. There is only one kind of basis: the linear basis map the input `x` in the `[a,b]` interval.

2.3.2 Approximation Techniques

We have tested some mapping techniques from the polynomial in monomial form and the Bernstein polynomial.

Support in $[0,1]$ and $N = M$

The simplest scenario is if we have a support in $[0,1]$ and the degree of the input and output polynomials are the same.

- We can provide a closed form formula to perform the conversion of the coefficients from the monomial ones to the Bernstein ones. This formula is $\beta_i = \sum_{j=0}^{i} \frac{\binom{i}{j}}{\binom{N}{j}} a_j$. We have a $\mathcal{O}(n^2)$ cost due to two inner loops.
- We can multiply the vector of Bernstein coefficients b and a conversion matrix M to obtain the monomial coefficient a as $a = Mb$; in this way we can find the Bernstein coefficient as $b = M^{-1}a$. We have a $\mathcal{O}(n^3) + \mathcal{O}(n^2)$ cost due to the inversion of M and two inner loops for the product; the inversion of the matrix is unstable for big order of N. This basis change matrix is obtained from these steps: the Bernstein basis are defined as $B_{i,N}(x) = \binom{N}{i} x^i (1-x)^{N-i}$;

we can expand this in $B_{i,N}(x) = \binom{N}{i} x^i \sum_{j=0}^{N-i} \binom{N-i}{j} (-1)^j x^j$ and then obtain $B_{i,N}(x) = \binom{N}{i} x^i \left[\sum_{j=0}^{N-i} \binom{N-i}{j} (-1)^j x^j \right]$; we can now change $i = k + j$ and obtain $B_{i,N}(x) = \sum_{j=i}^{N} \left[\binom{N}{i} \binom{n-i}{j-i} (-1)^{j-i} \right] x^j$; at this point we can define the matrix as $M_{i,j} = \begin{cases} \binom{n}{j}\binom{n-j}{i-j}(-1)^{i-j} & \text{se } i \geq j \\ 0 & \text{se } i < j \end{cases}$.

2.3.3 Functionality

This part of the library allows some operations that are independent from the domain model.

2.3.4 Distance

In the system there's the possibility of testing the distance between two given polynomials and an interval $[a, b]$. This is useful if we test the polynomial P_m in monomial form and its Bernstein approximation P_b.

The distances implemented are the three most used types of norm, that consider the whole interval. We have considered the following:

- L_1: It measures the global mean error and it's obtained as $\int_a^b |P_m(x) - P_b(x)| dx$.
- L_2: It penalizes the big error in the whole interval $[a, b]$ and it's obtained as $\sqrt{\int_a^b (P_m(x) - P_b(x))^2 dx}$.
- L_∞: It represents the biggest error in the interval between the two polynomials and it's obtained as $\max_{x \in [a,b]} |P_m(x) - P_b(x)|$.

Each of these norms, except for L_∞, is calculated with a numerical integration: it is a family of techniques to approximate the value of a definite integral without calculating the primitive of the function. This is useful when it is difficult to derive the primitive of the function or when the function isn't known in the analytical form.

In this case we used a specific Newton-Cotes formula, the Simpson's rule.

Visualization

There is an abstract class that allows us to plot a polynomial with a given precision and in a given interval.

3 Experiments

3.1 Monomial Approximation

Support in $[0,1]$ and $N = M$

We have evalueted the two techniques described in Section 2.3.2. For both methods the test include the same polynomial expressed in monomial form, i.e. $p_n(x) = \sum_{i=0}^{n} a_i x^i$, and differents orders n. These experiment are in this Main.java.

The results are allign with the limition of the `double` type described in Section **??**.

If we look the first method, i.e. the direct trasformation of the monomial coefficients to Bernstein coefficients, the results confirm that the method is exact the two polynomial, the approximator and the approximatee are the same. We can see that the error is in the order of 10^{-16}: if degree is 25 the L_1 norm error is 2×10^{-15}; if the degree is 1029 the error is $L_1 = 9.7 \times 10^{-15}$. If we increse firther the degree by one unit, the system fail and we obtain a NaN, exactly as described in Section **??**.

If we look to the method that use the basis matrix to compute the Bernstein coefficients, the thinks change drastically. Already for a small degree, like 10, we have an error of $L_1 = 1.84 \times 10^{-14}$; if we increase the degree to 35 we have $L_1 = 2.04 \times 10^{-2}$; if the degree is grether than 50 the problem began intractable and we obtain an error of $L_1 = 1.7 \times 10^5$.